Smart Warehouse Management and Pick-to-Light System: Design Document

Senior Capstone Project · Fathalla International Applied Technology School · Version 1.0 · 28 October 2026

Team: Person A (server and hardware) and Person B (Android app and admin dashboard)

1. Introduction

1.1 Background

Warehouses that supply branches, such as outlets in Smouha and El Asafra in Alexandria, usually pick orders from paper lists. Workers search for products, shelf stock is counted by hand, and nobody can see at a glance what is available or where an order stands. The result is slow picking, wrong items, and stock figures that drift away from reality.

1.2 Goal

Build a system in which an admin creates and assigns orders from a dashboard, each worker is guided to the right shelf by a floor-map route and a light on the shelf, and shelf stock stays accurate as work happens.

1.3 Objectives

- Track stock per shelf with an accurate available quantity at all times.
- Let orders be picked in parallel, with no order waiting on another to free a shelf.
- Keep assignment under the admin's control: the admin chooses the shelves, the workers and the worker for each order line.
- Guide picking with a map path, pick-to-light LEDs and barcode confirmation.
- Register inbound stock from the Android app by scanning the product barcode.
- Confirm the physical handoff of every outbound order at the loading dock.

1.4 Scope

In scope: outbound orders, inbound stock, shelf placement, per-shelf inventory, floor-map routing, pick-to-light, one admin dashboard and one Android app, for a single warehouse.

Out of scope: branch self-submission of orders (the admin enters them), automatic worker assignment or load balancing, real-time push to the dashboard, route optimisation across stops, live indoor positioning, a map editor, and multiple warehouses or multiple admins.

1.5 Terms

Term	Meaning
Carton	The unit for all stock quantities.
OnHand	Cartons physically on a shelf.
Committed	Cartons promised to unfulfilled order lines.
Available	OnHand minus Committed.
Order line	One shelf, one product and a quantity, assigned to one worker.
Anchor	The shelf a worker says they are standing next to, used as the start of a route.
Dock	The loading dock where finished orders are handed off.

2. Solution overview

2.1 Components

Component	Technology	Purpose
Backend API	ASP.NET Core Web API, EF Core	Business rules, stock, routing, device endpoint
Database	SQL Server	Shelves, stock, orders, map graph
Admin dashboard	ASP.NET MVC (PC)	Orders, assignment, inventory, inbound placement
Worker app	Android	Tasks, scanning, floor-map route, inbound entry, handoff

Component	Technology	Purpose
Shelf LED modules	ESP32 / Arduino	Light the shelf to pick from

2.2 Roles

- Admin: exactly one per warehouse. Enters orders, picks shelves, assigns lines to workers, assigns shelves to inbound stock, cancels and reassigns, monitors inventory.
- Worker: logs in by email, sees only their own assigned lines, picks, follows the route, confirms handoff, and enters inbound stock.

2.3 Outbound workflow

1. The admin creates an order: branch, deadline, and lines. Each line names a shelf, a product, a quantity in cartons and the worker who will pick it.
2. The system checks that every line is covered by available stock. If all pass, it commits the stock in one transaction. If any fail, nothing is committed and the admin sees which shelf is short.
3. Each assigned worker is notified and sees their lines in the app.
4. The worker taps their position on the map (the shelf they are next to), receives a path to the next shelf in rack-then-level order, and the shelf LED lights.
5. At the shelf the worker scans the product to confirm the pick. Shelf stock drops and the LED turns off.
6. After the last pick the app shows the path to the loading dock. The worker drops the goods, and confirms the handoff.
7. When every line of the order is handed off, the order is complete.

2.4 Inbound workflow

1. A worker opens the inbound screen, scans the product barcode, and enters the cartons and the receipt voucher number.
2. The admin assigns a shelf from the dashboard: the designated shelf for that product type, or the closest empty shelf if the designated one is full.
3. The worker places the cartons and confirms. Stock is counted at that moment.

3. Requirements

3.1 Functional requirements

1. The system shall keep OnHand and Committed per shelf and derive Available.
2. The admin shall create outbound orders with lines on specific shelves and a worker assigned to each line.
3. The system shall reject an order if any line exceeds its shelf's available stock, and shall leave no stock committed when it does.
4. Orders shall be processed in parallel with no queueing or shelf locking.
5. Workers shall confirm picks by scanning, and each pick shall reduce OnHand and Committed together.
6. The admin shall cancel an order only before its first confirmed pick, releasing all committed stock.
7. The admin shall reassign a line to another worker until that line is picked.
8. The app shall show a floor map with a path from the worker's anchor to the next shelf and finally to the dock.
9. The shelf LED for a pending line shall light and shall turn off when the pick is confirmed.
10. Workers shall register inbound stock by scanning the product barcode and entering cartons and voucher number.
11. The admin shall assign shelves to inbound stock, and the dashboard shall show which shelves are empty or full.
12. Workers shall confirm handoff at the dock, and the order shall complete when all its lines are handed off.
13. Only the assigned worker may pick or hand off a line.

3.2 Non-functional requirements

- Correctness over speed: stock figures must never go wrong, even when a write fails partway.
- Simplicity of use: screens designed for workers with limited reading time, with large targets and few steps.
- Responsiveness: the dashboard and LEDs refresh by polling within a few seconds.
- Maintainability: layered backend (controller, service, repository) with one place for each rule.

3.3 Assumptions and constraints

- One admin per warehouse, so there are no concurrent order creators.
- Every shelf holds one product at a time, and shelves are grouped by product type.
- All quantities are in cartons.
- The warehouse has Wi-Fi coverage for the phones and the LED controllers.
- Workers name their position by selecting a shelf, since there is no indoor positioning.

4. System architecture

4.1 Overview

The backend API is the single source of truth. The admin dashboard, the Android app and the LED controllers are all clients of the same API over HTTP. The dashboard is a server-rendered ASP.NET MVC application that calls the API like any other client.

4.2 Backend layers

- Controllers: authentication, role checks, request and response shapes.
- Services: every business rule, including the order-creation transaction, pick, cancel, inbound and routing.
- Repositories: data access through EF Core.

4.3 Communication

- Dashboard: refreshes by polling.
- LED controllers: each ESP32 polls an endpoint for the LEDs it should light.
- Workers: receive a notification when lines are assigned to them, and the app polls for task changes.
- Floor map: the API serves the map image and computes paths; the phone only draws them.

5. Data design

5.1 Entities

The full column-level schema and endpoint list are in the ER Schema and API Specification v2.0. In summary:

- User, Branch, ProductType, Product (piece and carton barcodes).
- Shelf: rack, level, designated product type, current product, max capacity, OnHand, Committed, map node, LED device and index.
- Order and OrderLine, where each line carries its shelf, product, cartons, assigned worker, status and handoff time.
- InboundOrder: voucher number, product, cartons, target shelf, status, timestamps.
- MapNode and MapEdge: the warehouse graph.

5.2 Stock model

Each shelf has two counters. Available = OnHand minus Committed. A new order is checked against Available. At a confirmed pick both counters fall by the picked cartons, so Available is unchanged by the pick itself and is correct at every moment. A product's total on the dashboard is the sum across its shelves.

5.3 Integrity and concurrency

The design deliberately avoids locking. Instead:

- Order creation validates every line first and writes only if all pass.
- The write runs in one database transaction. For each line a conditional update raises Committed only if enough is still available. If any update affects no rows, everything rolls back and the API returns a conflict naming the short shelf.
- Picks and cancels also run in single transactions.

Because there is one admin, two people never create orders at once. The transaction is still needed: a multi-line write can fail partway for ordinary reasons such as a timeout, an exception, a dropped connection or a double submit, and without it the earlier lines would stay committed to an order that does not exist.

6. Functional design

6.1 Order lifecycle

Status	Meaning	Next
Assigned	Created, no pick confirmed yet	InProgress, Cancelled
InProgress	At least one line picked	AwaitingHandoff
AwaitingHandoff	All lines picked, not all handed off	Completed
Completed	Every line handed off	none
Cancelled	Cancelled before the first pick	none

6.2 Assignment

Assignment is entirely the admin's decision. The admin picks the shelf for each line, using the dashboard's view of which shelves hold the product and how much is available, and picks the worker for each line. There is no automatic balancing. A pending line can be reassigned to another worker at any time.

6.3 Picking and handoff

A worker's route covers only their own remaining lines, in rack-then-level order, then the dock. Handoff is confirmed per worker per order: after a worker's last pick they walk to the dock, confirm, and their lines are marked handed off. The order completes when all its lines are handed off. Handoff never changes stock, which was already reduced at each pick.

6.4 Cancel and reassign

Cancel is allowed only while the order is Assigned, that is, before any confirmed pick. All committed stock is released in one transaction. After the first pick, cancel is refused.

6.5 Inbound and shelf placement

The worker enters inbound stock but it does not count until placement is confirmed. The admin assigns the shelf: the designated shelf for the product's type, or the closest empty shelf if that one is full, where closest means shortest path on the map graph. A shelf takes

its product when stock is first placed on it and releases it when it becomes empty again, which enforces one product per shelf.

6.6 Floor map and routing

The floor map is an exported image plus a graph held in the database. Nodes carry pixel coordinates on the image and a type (aisle point, shelf anchor, dock). Edges connect neighbouring nodes and are weighted by their length. Every shelf points at the node in front of it. When the worker selects their anchor shelf, the API runs Dijkstra over the in-memory graph and returns the path legs, each as a list of points. The phone scales the image to its screen and draws the points. After each pick the app asks again, using the shelf just picked from as the new anchor.

6.7 Pick-to-light

Each shelf level has an LED connected to a microcontroller. The controller polls the API and lights the LEDs for the next pending line of each active worker. When a pick is confirmed the LED turns off. With parallel orders several LEDs can be on at once.

7. Interface design

7.1 Android worker app

- Login by email.
- My tasks: the worker's own lines, grouped by order, with deadlines.
- Floor map: the map image, the anchor selector, the path to the next shelf, then to the dock.
- Pick: shelf code, product, cartons, and a scan button to confirm.
- Handoff: confirmation at the dock.
- Inbound: scan the product barcode, enter cartons and voucher number, then later confirm placement.

7.2 Admin dashboard (ASP.NET MVC)

- Shelves and inventory: per-shelf OnHand, Committed, Available, empty and full markers, and per-product totals.
- Create order: branch, deadline, and lines with shelf, product, cartons and worker.
- Orders: list and detail with line status, cancel and reassign.

- Inbound queue: pending records and shelf assignment.
- Workers: the list used when assigning lines.

7.3 API

The endpoint list, request and response shapes and error codes are in the ER Schema and API Specification v2.0.

8. Hardware design

The demo uses a wooden scale model of the warehouse with four to five shelf levels. Each level has an LED wired to an ESP32 or Arduino-class controller with Wi-Fi. Each controller is configured with a device id and polls the API for the list of its LEDs to light. A shelf record stores the device id and the LED index. Pin assignments, power and enclosure are detailed by Person A during build.

9. Security

- Email login with bearer-token authentication.
- Role checks on every endpoint: admin-only and worker-only operations are separated.
- A worker can read, pick and hand off only lines assigned to them; anything else returns forbidden.
- LED controllers use a device endpoint that exposes only LED state, never order data.
- Passwords are stored as salted hashes.

10. Testing plan

Check	Expected result
Two orders on one shelf exceed Available	The second is rejected, and none of its stock stays committed
Three-line order with a short third line	Committed is unchanged on the first two shelves
Pick confirmed	OnHand and Committed each fall by the cartons, Available is unchanged

Check	Expected result
Cancel before first pick	Committed is restored
Cancel after first pick	Refused
Worker picks a line assigned to someone else	Forbidden
Route from any anchor	Remaining shelves in rack-then-level order, ending at the dock
Inbound placed on an empty shelf	The shelf takes the product; emptying it clears the product
Graph connectivity	Every shelf anchor and the dock are reachable
LED behaviour	LED lights for a pending line and turns off on pick

Testing layers: unit tests for the service rules, API tests against a test database, a hardware bench test of LED polling, and a full rehearsal of the demo flow.

11. Implementation plan

Phase	Work	Owner
1	Schema, migrations, seed data, authentication	Person A
2	Shelf and inventory endpoints, order creation transaction, cancel, reassign	Person A
3	Worker endpoints, pick and handoff, inbound	Person A
4	Map graph, routing endpoint, device endpoint	Person A
5	LED controllers and the scale model	Person A
6	Android app: tasks, pick, map, inbound, handoff	Person B
7	Admin dashboard: inventory, create order, orders, inbound queue	Person B
8	Integration, rehearsal and polish	Both

Phases 6 and 7 can start against the API contract before phases 3 and 4 are finished.

12. Demonstration plan

A laptop runs the admin dashboard, an Android phone runs the worker app, a display screen shows the dashboard, and the wooden model shows the shelf LEDs. The demo walks through: an inbound entry and placement, creating an order with lines for two workers, an order rejected for insufficient stock, the map route and LED during a pick, handoff at the dock, and the inventory figures updating on the dashboard.

13. Limitations and future work

- Branches submit requests themselves instead of the admin entering them.
- Real-time updates over SignalR instead of polling.
- Automatic worker assignment and balancing as an optional aid to the admin.
- Reordering stops to shorten the walk.
- Live indoor positioning, for example with beacons, replacing the anchor selection.
- A map editor so the admin can change the layout.
- Support for several warehouses and several admins, which would require explicit concurrency control on order creation.

14. Design evolution

After the panel review and again in late October the design was simplified. The earlier version locked shelves and queued orders that needed busy shelves, balanced work across workers automatically and avoided live navigation. It now has no locking or queue: orders run in parallel and are protected by the available-stock check and a single transaction. Assignment moved to the admin, who now assigns each line to a worker. Workers got inbound entry in the app and a floor-map route from their anchor shelf through the pick list to the loading dock.

15. Conclusion

The system keeps stock accurate with two simple counters, lets orders run in parallel, leaves decisions with the admin, and guides workers with a route, a light and a scan. Its rules are few and testable, and the limits that remain are stated openly above.

Appendix A: Map preparation

1. Export the floor-map PDF as an image and record its pixel width and height.
2. Place a node at every aisle corner and junction, a shelf-anchor node in front of every shelf, and one dock node. Record each node's X and Y on the image.
3. Connect neighbouring nodes with edges along walkable aisles only. Edge weights are computed from node coordinates.
4. Link each shelf to its anchor node.
5. Seed the nodes and edges into the database.
6. Validate: every shelf anchor and the dock must be reachable from every other node. A missing edge is the most likely defect, and the routing endpoint reports the unreachable node.

Appendix B: Open items and risks

- Handoff is confirmed per worker, and an order completes when all lines are handed off. This decision is confirmed.
- The map graph is prepared after the documentation is final and before implementation starts, and needs the exported image to measure.
- The app needs network access throughout a task, since routes are computed on the server.